DAY 09 / 11 · WEB DEV IN 11 DAYS · GUIDED

The show page

Views, details and My List

ShowShelf after today: **a page for every show, with its cast and seasons, and a page for My List**

TODAY

The plan for the next 90 minutes

Minutes	Topic
10	Recap
12	Three views, one page
12	One show, with its cast and seasons
15	Filling the show view
12	Back, loading and “not found”
14	My List remembers more
15	This afternoon in the Lab

RECAP · DAY 8

Five quick questions

1. What does await do, and where can you use it?
2. await fetch(url) gave a response. What's the second step?
3. A search answers [{ score, show }]. The first show's name?
4. The four states of a page that loads data?
5. response.ok is false. What happened, and what did we do?

VIEWS

Three screens, one page

A **view** is one screen of an app: home, show, My List.

state.view says which one is showing. **render()** hides the others.

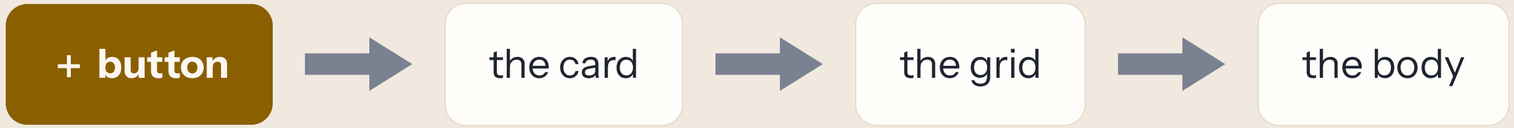
```
// hidden, unless it's the chosen one
cartPanel.hidden = state.panel !== "cart";

// to switch: change the state, then draw
state.panel = "cart";
render();
```

!== means “is not”. Read it aloud: hide the cart panel unless the panel is cart.

EVENTS

A click bubbles up




```
deleteButton.addEventListener("click", (event) => {
  event.stopPropagation(); // not also a click on the row
  deleteEmail(email);
});
```

A click counts for the button, then for everything around it. **event.stopPropagation()** stops it there.

TVMAZE

One request, three kinds of data

`/shows/169`
`?embed[]=cast&embed[]=seasons`

One show by its id, with its cast and seasons inside **_embedded**.

An id that doesn't exist: **404 Not Found**.

```
{
  "id": 169,
  "name": "Breaking Bad",
  ...
  "_embedded": {
    "cast": [
      { "person": { "name": "Bryan Cranston" },
        "character": { "name": "Walter White" } },
      ...
    ],
    "seasons": [
      { "number": 1, "episodeOrder": 7,
        "premiereDate": "2008-01-20" },
      ...
    ]
  }
}
```

SAFETY

Text from the internet goes in as text

innerHTML

Treats the text as HTML. Code hidden in it **runs on your page**.

textContent

Shows the text as text, whatever is in it. **Always safe.**

```
// a "summary" with a trap inside
heading.innerHTML = '<img src="x" onerror="document.body.style.background = \'red\'">';
```

The rule: text that comes from anywhere else goes in with textContent, never innerHTML.

NESTED DATA

One level at a time

member.person.name: into member, into person, then the name.

A list is built like the grid: empty it, then **one new element per item**.

member.name is undefined: the name is one level deeper.

```
const team = [
  { person: { name: "Ana" },
    role: { name: "goalkeeper" } },
  { person: { name: "Boris" },
    role: { name: "captain" } },
];

teamList.replaceChildren();
for (const member of team) {
  const item = document.createElement("li");
  item.textContent = member.person.name
    + " plays " + member.role.name;
  teamList.append(item);
}
```

SMALL HELPERS

The same lines again? Make a function.

Each season is a table row: a **<tr>** with three **<td>** cells.

Three lines per cell, three cells per row. One small function makes it one line.

```
function createCell(text) {
  const cell = document.createElement("td");
  cell.textContent = text;
  return cell;
}

// append can take several things at once
row.append(
  createCell("Friday"),
  createCell("Fleabag"),
  createCell("2 episodes")
);
```

WHEN THINGS GO WRONG

Back, loading and “not found”

Back

Only a change of state: the home view, then render(). Nothing loads again.

Loading

The shape of loadShows: a message before await, try / catch around it.

Not found

A 404 is an answer, not a broken connection: give back null.

```
if (name === "") {
  console.log("Nobody to greet.");
  return; // the function ends here
}
console.log("Hello, " + name + "!");
```

MY LIST

Save what a card needs

```
{ id: 431, name: "Friends", image: null,
  genres: ["Comedy", "Romance"],
  rating: 8.8 }
```

No request per show, and it works offline.

```
const saved = [
  { id: 7, name: "Chernobyl" },
  { id: 10, name: "Band of Brothers" },
];

saved.includes(7); // false
saved.some((item) => item.id === 7); // true
```

some: does at least one item pass the test? **includes** compares whole objects with 7.

REMEMBER

One state, one render(). The card, the show view and My List always agree.

THIS AFTERNOON IN THE LAB

Eleven steps, about three hours

Steps 1–3

The show view

1 Three views, one page

2 One show, with cast and seasons

3 Fill in the show

Steps 4–6

Cast, seasons, back

4 The cast

5 The seasons table

6 The back button

Steps 7–8

When it goes wrong

7 Loading and errors

8 Show not found

Steps 9–11

My List

9 My List remembers more

10 The My List view

11 My List from the show view

Milestone: open a show, add it from the show view, go back: the tick is there, and My List has it.

STUCK?

In this order

1

Read the message

The “Not yet” box says what to fix.

2

Console

A red line names the problem and the line number.

3

Hint

Two per step. The second is nearly the answer.

4

Answer

Compare it with yours. Find the difference.

5

Ask

Your teacher, or the person next to you.

Step 7 is the trickiest today: scroll up to loadShows and copy its shape.

END OF THE DAY

Before you go

1. homeView.hidden = state.view !== "home": when is home hidden?
2. Why stopPropagation on the + button?
3. Why does My List save small objects, not only ids?

FINISHED EARLY?

Extra challenges

A link for every show · Cast photos · The average rating of My List

TOMORROW

A taste of React and of servers

A real React project and a real server, then small versions of both in the Lab.